



UNIVERSITÀ DI PISA

Dipartimento di Ingegneria dell'Informazione

Master's Degree in Artificial Intelligence and Data Engineering

Smart Health Monitoring System

Studente: Antonio Pepe

Matricola: 665417

Docenti: Prof. Giuseppe Anastasi

Prof. Carlo Vallati

Ing. Francesca Righetti

Anno Accademico 2025/2026

Contents

1	Introduzione	2
1.1	Dominio di applicazione	2
2	Schema architetturale	3
2.1	Protocollo di rete	3
2.1.1	Interfacce CoAP ed Encoding JSON	3
2.2	Configurazione dei dispositivi	4
2.3	Database MySQL	4
3	Modello di Machine Learning	5
3.1	Funzione principale	5
3.2	Dataset	5
3.3	Addestramento	5
3.3.1	Accuratezza del Modello	6
3.3.2	Inferenza sul Dispositivo	6
4	Applicazione Cloud	7
4.1	Backend	7
4.2	Closed-Loop e Logica Adattiva	7
4.3	Interfaccia Utente	7
4.4	Interazioni Hardware	8
4.4.1	Pulsante: Attivazione del Meccanismo Adattivo	8
4.4.2	LED: Indicatori di Stato del Paziente	8
5	Valutazione e stress test	9
5.1	Scenario Node Failure	9
5.2	Meccanismo Adattivo	9
5.2.1	Attivazione via Pulsante Hardware	9
5.2.2	Attivazione via Interfaccia Utente	9
5.3	Grafana	10
5.3.1	Condizione normale e sotto stress	10
5.3.2	Query Grafana	10

Chapter 1

Introduzione

1.1 Dominio di applicazione

Il dominio di applicazione scelto per questo progetto è stato lo "SmartHealth". Il progetto riguarda un sistema di monitoraggio paziente. L'obiettivo è stato misurare valori come temperatura corporea, battito cardiaco e la saturazione attraverso sensori in modo tale da prevedere situazioni di pericolo o di stato di emergenza così da, attraverso un attuatore, notificare le condizioni del paziente.

Il mio progetto si è basato su alcuni paper che ho trovato su google scholar:

1. **The Internet of Things for Health Care: A Comprehensive Survey** (S. M. R. Islam, D. Kwak, M. H. Kabir, M. Hossain and K. S. Kwak — *IEEE Access*, 2015)

Fornisce una panoramica sui requisiti di campionamento dei sensori biomedici e formalizza le architetture di rete a tre livelli da Sensore → Gateway → Cloud.

2. **Self-Aware Early Warning Score System for IoT-Based Personalized Healthcare** (I. Azimi, A. Anzanpour, M. Rahmani-Andebili, et al. — *FedCSIS*, 2016)

Questo lavoro descrive la digitalizzazione e l'automazione dei protocolli clinici EWS (*Early Warning Score*) e MEWS all'interno di ecosistemi IoT. Spiega come mappare i dati combinati di temperatura, frequenza cardiaca e saturazione dell'ossigeno (SpO_2) in classi di rischio numeriche. Tale studio costituisce la base logica dell'algoritmo di machine learning.

Chapter 2

Schema architetturale

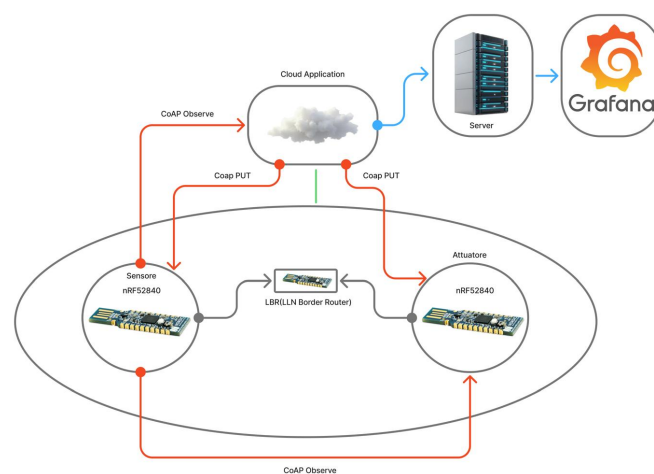


Figure 2.1: Schema architetturale dell'applicazione IoT.

Lo schema dell'applicazione è molto semplice. Abbiamo un Border Router implementato con un nRF52840. Al BR ho collegato due dongle fisici, sempre l'nRF52840, uno che svolge la funzione di sensore ed uno di attuttore. Il flusso di funzionamento segue le frecce in figura.

2.1 Protocollo di rete

Come protocollo di rete ho deciso di utilizzare CoAP (Constrained Application Protocol). E' un protocollo a livello applicazione basato su UDP, quindi più leggero, ma come punto a suo sfavore non è affidabile. In realtà per la situazione in cui ci troviamo quindi progettare reti per IoT, quindi dispositivi con risorse limitate, questo protocollo è perfetto. Inoltre il protocollo mette a disposizione vari metodi e varie funzioni, infatti abbiamo che il protocollo è valido sia lato client che lato server. Infine CoAP, per definizione, è un protocollo che mette a disposizione il resource observing: nel nostro caso abbiamo sensori/attuatori che espongono risorse che la Cloud Application deve cambiare.

2.1.1 Interfacce CoAP ed Encoding JSON

Per la comunicazione tra i nodi e la Cloud Application ho scelto il formato JSON (JavaScript Object Notation), ideale per dispositivi con risorse limitate perchè usa solo due tipi di dati (oggetti e array),

quindi è molto leggero. I nodi espongono specifiche interfacce CoAP che definiscono le API del sistema:

- **Nodo Sensore - Risorsa /vitals:** Interrogabile tramite metodo GET. Restituisce i parametri misurati e la classe di rischio elaborata localmente dal modello ML. Essendo observable, la Cloud Application e l'attuatore ricevono gli aggiornamenti in modo asincrono.
Payload: {"hr": %d, "body_temperature": %d, "spo2": %d, "risk": %d} [?]
- **Nodo Sensore - Risorsa /sampling:** Accetta richieste PUT per modificare dinamicamente il rate di acquisizione (es. da 5 a 20 secondi) per l'attivazione del meccanismo adattivo.
Payload: {"new_sr": %d}
- **Nodo Attuatore - Risorsa /threshold):** Espone un'interfaccia interrogabile tramite metodo GET e modificabile tramite PUT. Accetta richieste PUT dalla Cloud Application per aggiornare dinamicamente il valore di allerta memorizzato sul dispositivo.
Payload: {"threshold": %d}

2.2 Configurazione dei dispositivi

La configurazione dei dispositivi è stata la parte relativamente più complessa. Ho configurato i nodi come spiegato nella sezione del sistema architetturale. La configurazione dei nodi è stata fatta in modo statico attraverso un file python (`configuration_manager.py`):

```
# da configuration_manager.py
SENSORS_CONFIG = {
    "Sensori": {
        "id": 1,
        "sensor_ip": "fd00::f6ce:36b9:a760:ecea", # Ip del dongle che funge da
        "actuator_ip": "fd00::f6ce:36a6:c68f:cb04" # IP dell'attuatore fisico
    }
}
```

Listing 2.1: Configurazione del database e dei sensori in `configuration_manager.py`

2.3 Database MySQL

Ho utilizzato un database MySQL. Di seguito lo script di creazione del database e delle tabelle:

```
CREATE DATABASE IF NOT EXISTS SmartHealthIoT;
USE SmartHealthIoT;
CREATE TABLE IF NOT EXISTS Health_Measurements (
    id INT AUTO_INCREMENT PRIMARY KEY,
    sensor_id INT NOT NULL,                -- ID del sensore associato
    heart_rate INT NOT NULL,               -- Frequenza cardiaca (bpm)
    body_temperature INT NOT NULL,         -- Temperatura corporea (°C)
    spo2 INT NOT NULL,                    -- Saturazione dell'ossigeno (%)
    risk_score INT NOT NULL,               -- Classe di rischio (0-3)
    status VARCHAR(20) NOT NULL DEFAULT 'ONLINE', -- Stato del sensore
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,

    -- Indice per velocizzare le query di Grafana e della UI
    INDEX idx_sensor_timestamp (sensor_id, timestamp DESC)
);
```

Chapter 3

Modello di Machine Learning

3.1 Funzione principale

E' stato usato un modello di Machine Learning, più precisamente il Random Forest per generare, partendo da un dataset, più alberi decisionali, in base ai parametri vitali. Il modello ha collezionato dati riguardanti la temperatura, battito cardiaco e l'ossigenazione. Grazie a questi tre parametri sono riuscito ad addestrare il modello ed associare ad ogni situazione una classe di rischio. Come da progetto, si è fatto uso del tinyML, attraverso Google Colab, per inferire il modello.

3.2 Dataset

Il dataset utilizzato per l'addestramento del modello di Machine Learning è stato trovato su Kaggle. Nello specifico, è stato utilizzato il seguente dataset:

<https://www.kaggle.com/datasets/jacobhealth/health-status-dataset>

Il dataset descrive la relazione tra temperatura corporea, frequenza cardiaca e saturazione dell'ossigeno nel sangue (SpO₂). Lo stato di salute del paziente è rappresentato da un'etichetta numerica che va da 0 a 2, secondo la seguente classificazione:

- **0** – Nessun parametro fuori dalla norma rilevato;
- **1** – Il paziente è malato, ma la condizione non è in pericolo di vita;
- **2** – Il paziente è in pericolo di vita.

Nel dataset troviamo più di 5000 tuple.

3.3 Addestramento

L'addestramento del modello è stato effettuato utilizzando Google Colab, come si può vedere dal notebook `ML_model.ipynb` che si trova nella directory `Machine_Learning/` del progetto.

La procedura ha seguito cinque passi fondamentali.

1. **Primo passo:** caricamento del dataset e aggiornamento librerie.
2. **Secondo passo:** preprocessing. Le tre feature disponibili (temperatura, frequenza cardiaca e SpO₂) sono state selezionate come input del modello, mentre la colonna target è stata utilizzata come etichetta per la classificazione.

3. **Terzo passo:** scelta del modello. È stato scelto un algoritmo Random Forest, un classificatore che combina multiple decision tree per ottenere predizioni. La scelta è ricaduta su questo algoritmo perché è particolarmente adatto all'esecuzione su dispositivi embedded con risorse limitate, grazie alla possibilità di essere convertito in codice C efficiente tramite la libreria emlearn.
4. **Quarto passo:** addestramento e validazione attraverso Google Colab (80% addestramento e 20% validazione).
5. **Quinto passo:** esportazione per il deployment embedded. Il modello addestrato è stato convertito in codice C tramite la libreria emlearn. Il file generato, `vitals_classifier.h`, contiene la funzione `vitals_rf_model_predict()` che viene poi inclusa direttamente nel firmware del nodo sensore.

```
# 3. ADDESTRAMENTO RANDOM FOREST
# (Ottimizzato per Contiki-NG)
print("\nAddestramento del modello Random Forest sui dati reali di Kaggle...")
# Mantengo 5 alberi e profondità' 3 per non saturare
# la memoria RAM limitata dei nodi in Cooja
random_forest_model = RandomForestClassifier(
    n_estimators=5, max_depth=3, random_state=42
)
random_forest_model.fit(X_train, y_train)

# 4. VALUTAZIONE DELL'ACCURATEZZA
y_pred = random_forest_model.predict(X_test)
print("\n--- REPORT DI ACCURATEZZA TINYML ---")
print(classification_report(y_test, y_pred))
print(f"Gradi di rischio elaborati dal modello: "
      f"{random_forest_model.classes_}")
```

Listing 3.1: Addestramento del modello Random Forest in `ML_model.ipynb`

3.3.1 Accuratezza del Modello

Dopo l'addestramento, il modello Random Forest è stato valutato sul test set (20% dei dati, circa 1182 campioni). I risultati ottenuti sono riportati di seguito:

```
--- REPORT DI ACCURATEZZA TINYML ---
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	215
1	1.00	0.90	0.95	473
2	0.91	1.00	0.95	494
accuracy			0.96	1182
macro avg	0.97	0.97	0.97	1182
weighted avg	0.96	0.96	0.96	1182

Come si evince dal report, il modello raggiunge un'accuratezza complessiva del 96%.

3.3.2 Inferenza sul Dispositivo

Il file `vitals_classifier.h` viene incluso nel codice del nodo sensore, precisamente nel file `sensor.c`. A ogni ciclo di campionamento, il sensore esegue localmente il modello. I tre parametri vitali vengono inseriti in un array di feature e passati alla funzione di predizione, che restituisce il livello di rischio clinico. Questo valore viene poi esposto tramite la risorsa `CoAP/vitals`.

Chapter 4

Applicazione Cloud

4.1 Backend

Per quanto riguarda il backend, ma anche il frontend, tutto il sistema è stato scritto in Python. Ho preferito utilizzare un ambiente virtuale (venv) per isolare tutte le dipendenze. Il sistema cloud si trova nella cartella `Cloud_Application/`. All'interno di questa cartella troviamo due file che riguardano l'applicazione lato backend:

- `db_setup.sql`: Questo script SQL inizializza il database relazionale `SmartHealthIoT`. Crea la tabella `Health_Measurements` strutturata con i campi per i parametri vitali (`hr INT`, `body_temperature INT`, `spo2 INT`), la classe inferita (`risk INT`), lo stato di rete del nodo (`status VARCHAR`) e un timestamp (`CURRENT_TIMESTAMP`) generato automaticamente all'inserimento del record.
- `SmartHealthCloud.py`: Questo backend Python usa il meccanismo *Observe* di CoAP per ricevere in modo asincrono i dati dai sensore senza doverlo interrogare periodicamente. Quando arrivano i dati JSON, fa il parsing, salva le misurazioni nel database MySQL e, se il livello di rischio clinico è cambiato, invia un comando all'attuatore per aggiornare lo stato dell'allarme (verde, blu, rosso).

4.2 Closed-Loop e Logica Adattiva

Il sistema funziona in questo modo: il sensore raccoglie dati inerenti ai parametri vitali citati precedentemente (in realtà alcuni parametri vengono generati in modo randomico, come anche alla temperatura viene aggiunto un gap perché il sensore simula la sua temperatura interna). Successivamente, il sensore utilizza il modello Random Forest per fare inferenza e determinare, dai dati raccolti, la classe di rischio clinico. In seguito, il sensore invia la classe di rischio e i dati vitali tramite il protocollo CoAP alla Cloud Application e all'attuatore.

Il sistema di backend `SmartHealthCloud.py` dopo aver ricevuto i dati dal sensore via CoAP *Observe*, li salva nel database MySQL. L'attuatore, anche osserva il sensore in background, ma il cambiamento del LED viene gestito attraverso la richiesta PUT inviata dal Cloud su `/threshold`, eliminando ogni conflitto.

4.3 Interfaccia Utente

L'interfaccia utente è stata sviluppata attraverso la libreria Python Tkinter. Per avviare l'interfaccia è necessario attivare l'ambiente venv e spostarsi nella directory

`Cloud_Application/frontend/` ed eseguire il comando:

```
python SmartHealthUI.py
```


Fatto questo, si aprirà un pannello dove vengono visualizzati tutti i parametri attualmente misurati dal sensore e la classe di rischio inferita direttamente dal nodo sensore. Inoltre, è possibile controllare il *rate* di aggiornamento.

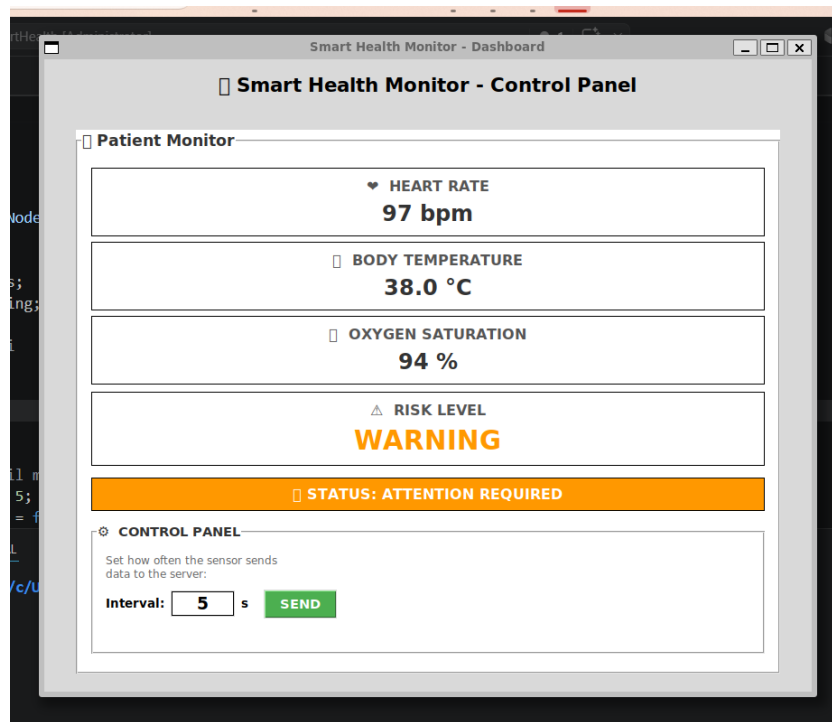


Figure 4.1: Screenshot interfaccia UI per SmartHealth

4.4 Interazioni Hardware

Il sistema SmartHealth utilizza i pulsanti e i LED disponibili sui nodi IoT dei dongle.

4.4.1 Pulsante: Attivazione del Meccanismo Adattivo

Il pulsante hardware sul nodo sensore è stato configurato per attivare il meccanismo adattivo.

La logica è implementata nel processo principale del sensore (file `sensor.c`). Quando il meccanismo adattivo è attivo, il periodo di campionamento viene aumentato da 5 a 20 secondi. Premendo nuovamente il pulsante, il sistema ritorna alla condizione operativa normale.

4.4.2 LED: Indicatori di Stato del Paziente

Nel progetto vengono utilizzati anche i LED. Come spiegato nella sezione closed-loop ogni volta che viene rilevato un cambiamento della classe di rischio, la cloud application notifica tramite CoAP all'attuatore.

- **Rischio 0 (normale):** LED verde, situazione stabile.
- **Rischio 1 (attenzione):** LED blu, il paziente è malato ma non in pericolo di vita.
- **Rischio 2 (emergenza):** LED rosso, il paziente è in pericolo di vita.

Chapter 5

Valutazione e stress test

5.1 Scenario Node Failure

Il sensore reale, collegato via USB al PC e flashato con il firmware `sensor.c`, può essere fisicamente scollegato. Quando questo accade, il nodo scompare dalla rete RPL e il backend cloud non riceve più notifiche CoAP. Il sistema rileva il failure attraverso il metodo `fetch_latest_measurements()` nel DB-Manager, che verifica periodicamente il timeout dinamico basato sul `sampling rate`. Quando il tempo trascorso dall'ultimo dato supera il timeout, viene inserito un record con status `SENSOR_MISSING`. Nell'interfaccia utente viene effettuato un controllo: quando il sensore non è raggiungibile, tutti gli elementi della dashboard vengono disabilitati. A livello di database, la tabella `Health_Measurements` include una colonna `status` che può assumere il valore `ONLINE` o `SENSOR_MISSING`. Quando un sensore smette di inviare dati per un certo periodo, il sistema aggiorna questo campo per segnalare il guasto, permettendo alla dashboard Grafana di mostrare lo stato del nodo.

5.2 Meccanismo Adattivo

Il sistema implementa un meccanismo adattivo che consente di modificare dinamicamente la frequenza di campionamento del sensore in risposta a condizioni di stress della rete. Questo meccanismo è attivabile attraverso due modalità complementari: pulsante hardware sul nodo sensore e interfaccia utente (UI).

5.2.1 Attivazione via Pulsante Hardware

Come descritto nella Sezione 4.4.1, il pulsante fisico presente sul dongle del sensore consente di commutare lo stato del meccanismo adattivo. Premendo il pulsante, il sistema alterna il periodo di campionamento tra 5 secondi (modalità normale) e 20 secondi (modalità adattiva).

5.2.2 Attivazione via Interfaccia Utente

Oltre al controllo hardware, è stata implementata una funzionalità direttamente nell'interfaccia grafica (file `SmartHealthUI.py`) che permette di simulare condizioni di stress di rete variando il *sampling rate* del sensore in tempo reale.

Nella UI è presente un pannello di controllo che espone un campo di input numerico. L'utente può inserire un valore (espresso in secondi) che rappresenta il nuovo intervallo di campionamento desiderato. Il backend cloud, attraverso CoAP, invia questo valore al nodo sensore che aggiorna dinamicamente il proprio rate di acquisizione dati.

Sul lato sensore, è stata implementata una risorsa CoAP dedicata `/sampling` che accetta richieste di tipo PUT per modificare il parametro `current_sampling_rate`.

5.3 Grafana

5.3.1 Condizione normale e sotto stress

In condizioni normali, il sensore campiona e invia i dati ogni 5 secondi. I grafici mostrano un flusso regolare e continuo. Si tratta del comportamento base del sistema, quello che ci si aspetta in una rete non congestionata.

Quando si preme il pulsante sul sensore, il meccanismo adattivo si attiva e il periodo di campionamento passa da 5 a 20 secondi. Sulla dashboard questo cambiamento è visibile immediatamente: i punti sul grafico si diradano, perché i dati arrivano con minore frequenza.

Inoltre, poiché è possibile modificare il sampling rate anche a intervalli inferiori a 5 secondi, dai grafici si osserva una densità di punti più alta e maggiormente ravvicinati, evidenziando così un carico di rete più elevato.



Figure 5.1: Grafici Grafana

5.3.2 Query Grafana

Query 1 - Trend dei Parametri Vitali

Questa query estrae e aggrega nel tempo i tre parametri fisiologici misurati (frequenza cardiaca, temperatura corporea e saturazione di ossigeno), calcolandone la media.

Query 2 - Livello di Rischio Clinico

Restituisce l'evoluzione temporale della classe di rischio.

Query 3 - Pacchetti Ricevuti al Minuto

Conteggia il numero di messaggi CoAP ricevuti dal cloud application in finestre di un minuto. Questa metrica quantifica l'impatto del meccanismo adattivo.

Query 4 - Stato del Sensore nel Tempo

Monitora il nodo IoT distinguendo tra stato ONLINE (funzionamento regolare) e stato SENSOR_MISSING (nodo non raggiungibile).